CSC 8607 – Introduction au deep learning

CI4 : Réseaux récurrents et génération de séquences

Objectifs : *Le but de ce TP est de mettre en pratique les concepts théoriques étudiés sur les réseaux de neurones récurrents (RNN, LSTM) et sur l'instabilité des gradients (vanishing/exploding gradients) pour résoudre un problème concret de modélisation générative : la* ***génération automatique de partitions musicales au format ABC****.*

Modalités de rendu : *L'évaluation se fera sur la base d'un rapport au format Markdown (*`rapport.md`*). Dans le même dépôt Git que lors des TPs précédents, créez un répertoire* `TP4` *et placez-y votre rapport, votre script Python (*`train_music.py`*), votre script Slurm (*`run_music.sh`*) ainsi que vos captures TensorBoard. Mettez à jour votre dépôt régulièrement.*

Remarque : Organisation des expériences : *Le jeu de données complet contient plus de 200 000 partitions musicales. Comme au TP3, nous travaillerons en deux étapes :*

- *En séance, vous travaillerez sur un* ***sous-ensemble de 10 000 partitions*** *afin de comparer rapidement l'entraînement d'un RNN classique (Elman) et d'un LSTM, ainsi que l'effet du Gradient Clipping.*
- *En fin de séance, vous lancerez un entraînement plus long avec sauvegarde du meilleur modèle (Early Stopping) via* `sbatch` *pour générer et écouter vos propres compositions musicales.*

Objectifs

- Manipuler et vectoriser des séquences textuelles de longueur variable (vocabulaire, indexation, *padding* et troncature).
- Construire un `Dataset` auto-régressif (prédiction du caractère suivant par décalage temporel).
- Implémenter et comparer une architecture `nn.RNN` et une architecture `nn.LSTM` précédées d'une couche d'`Embedding`.
- Mesurer la norme des gradients au cours de la *Backpropagation Through Time* (BPTT) et appliquer le *Gradient Clipping*.
- Générer de nouvelles séquences musicales en contrôlant le compromis diversité/cohérence via l'échantillonnage par **température**.

Exercice 1 : Exercice 1 : Exploration et Prétraitement des partitions ABC (~35mn)

Les données d'entraînement et de validation sont stockées sur le cluster dans le dossier `/mnt/ssd/irishman`. Chaque fichier (`train.json` et `validation.json`) est au format JSON et contient une liste de dictionnaires. Chaque élément possède la clé `"abc notation"` contenant une partition musicale complète sous forme de chaîne de caractères.

La notation ABC est un format texte compact utilisant des caractères ASCII pour représenter des mélodies (notamment le répertoire traditionnel irlandais). Elle débute par des lignes d'en-tête comme `X:` (numéro), `L:` (durée de note par défaut), `M:` (métrique, ex: `6/8` ou `4/4`) et `K:` (tonalité, ex: `G` ou `Dmaj`), suivies des notes (`A-G`, `a-g`), altérations et barres de mesure (`|`, `||`, `:|`).

Étape 1 : Chargement et découverte du jeu de données

Question 1.a. Créez un répertoire `TP4` dans votre dépôt Git et commencez un script `train_music.py`. Complétez le code ci-dessous pour charger les données, afficher les statistiques de base et inspecter la première partition.

```python
import os
import json
import time
import random
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torch.utils.tensorboard import SummaryWriter
from tqdm import tqdm

# Reproductibilité
torch.manual_seed(42)
random.seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

DATA_DIR = "/mnt/ssd/irishman"

def load_data(file_path):
    with open(file_path, "r", encoding="utf-8") as f:
        return json._____(f)

train_data = load_data(os.path.join(DATA_DIR, "train.json"))
val_data   = load_data(os.path.join(DATA_DIR, "validation.json"))

print(f"Nombre de chansons (Train) : {len(train_data)}")
print(f"Nombre de chansons (Val)   : {len(val_data)}")
print("\n--- Exemple de partition (Train[0]) ---")
print(train_data[0]["abc notation"])
```

✍️ Action et Rapport (`rapport.md`) :

1. Exécutez ce code et indiquez dans votre rapport le nombre exact de partitions présentes dans **`train.json`** et dans **`validation.json`**.
2. Copiez le texte de la première partition (**`train_data[0]["abc notation"]`**) et collez-le dans le lecteur interactif ABC Player pour voir la partition et l'écouter. En observant le texte brut, identifiez et expliquez le rôle des métadonnées d'en-tête présentes sur les premières lignes avant le début des notes.

Étape 2 : Construction du vocabulaire et Padding

Un réseau de neurones ne pouvant manipuler que des tenseurs numériques, nous allons travailler au **niveau caractère** (*character-level language modeling*). Pour regrouper plusieurs partitions dans un même *mini-batch* (tenseur 2D de forme **`(batch_size, seq_len)`**), toutes les séquences d'un batch doivent partager exactement la même longueur : nous allons donc tronquer les partitions trop longues et compléter les partitions trop courtes avec des espaces (*padding*).

Question 1.b. Complétez le code suivant pour extraire la liste triée des caractères uniques, construire les dictionnaires de correspondance (caractère -> indice), et normaliser la longueur des séquences à **`SEQ_LENGTH = 500`**.

```
# 1. Extraction déterministe (triée) des caractères uniques
all_text = "".join(song["abc notation"] for song in train_data + val_data)
unique_chars = sorted(list(_____(all_text)))
vocab_size = len(unique_chars)
print(f"Taille du vocabulaire : {vocab_size} caractères uniques")

# 2. Mappings caractère  index
char_to_index = {char: idx for idx, char in enumerate(_____________)}
index_to_char = {idx: char for idx, char in enumerate(_____________)}

def vectorize_string(s):
    return [_____________[char] for char in s]

# 3. Statistiques de longueur et Padding
lengths = [len(song["abc notation"]) for song in train_data]
max_len = max(lengths)
avg_len = sum(lengths) / len(lengths)
print(f"Longueur moyenne : {avg_len:.1f} | Longueur max : {max_len}")

SEQ_LENGTH = 500  # Longueur fixe choisie (environ 2x la longueur moyenne)

def pad_or_truncate(sequence, max_length=SEQ_LENGTH):
    if len(sequence)
```

✍️ À mettre dans votre rapport :

1. Combien y a-t-il de caractères uniques (**`vocab_size`**) ? Quelle est la longueur moyenne et la longueur maximale d'une partition dans le jeu d'entraînement ?
2. Pourquoi est-il indispensable d'appliquer **`sorted(...)`** sur le **`set`** des caractères uniques avant de créer **`char_to_index`** ? Que se passerait-il si vous sauvegardiez les poids de votre modèle aujourd'hui et que vous relanciez votre script demain pour générer de la musique sans ce **`sorted()`** ?
3. Pourquoi avons-nous choisi de tronquer à **`SEQ_LENGTH = 500`** (environ deux fois la longueur moyenne) plutôt que d'aligner toutes les chansons sur la longueur maximale **`max_len`** du dataset ? Quel serait l'impact d'un padding jusqu'à **`max_len`** sur le temps de calcul, la mémoire GPU et l'apprentissage du modèle ?

Exercice 2 : Exercice 2 : Dataset PyTorch auto-régressif et Modélisation (~35mn)

La génération de séquences repose sur un apprentissage **auto-régressif** (*Many-to-Many* synchrone causal) : à chaque pas de temps t, le réseau reçoit le caractère x_t (ainsi que l'état caché h_{t-1} résumant x_1, ..., x_{t-1}) et doit prédire le caractère suivant y_t = x_{t+1}. Ainsi, pour une séquence de longueur L, l'entrée est constituée des L-1 premiers caractères et la cible (*target*) est la même séquence décalée d'un cran vers la gauche (les L-1 derniers caractères).

Étape 1 : Création du `MusicDataset`

Question 2.a. Complétez la classe **`MusicDataset`** ci-dessous pour préparer les séquences d'entrée et de sortie décalées d'un pas de temps :

```
class MusicDataset(Dataset):
    def __init__(self, raw_data, seq_length=SEQ_LENGTH, subset_size=None):
        if subset_size is not None:
```

```
            raw_data = raw_data[:subset_size]

        self.data = [
            vectorize_string(pad_or_truncate(song["abc notation"], seq_length))
            for song in raw_data
        ]

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        seq = self.data[idx]
        # Entrée : tous les caractères sauf le dernier
        x = torch.tensor(seq[____], dtype=torch.long)
        # Cible : tous les caractères sauf le premier (décalage de +1)
        y = torch.tensor(seq[____], dtype=torch.long)
        return x, y
```

✍️ À mettre dans votre rapport : Si l'on fixe **`SEQ_LENGTH = 500`** et **`BATCH_SIZE = 128`**, quelles sont les dimensions exactes (*shapes*) des tenseurs **`x`** et **`y`** renvoyés par une itération du **`DataLoader`** ? Pourquoi utilise-t-on le type **`torch.long`** (entiers 64 bits) plutôt que **`torch.float32`** pour ces tenseurs ?

Étape 2 : Architecture récurrente (RNN vs LSTM)

Nous allons maintenant construire une classe modulable **`MusicModel`** capable d'utiliser au choix un RNN classique d'Elman (**`nn.RNN`**) ou un LSTM (**`nn.LSTM`**). L'architecture comporte trois blocs :

1. Une couche d'Embedding (**`nn.Embedding`**) projetant chaque indice de caractère dans un espace continu de dimension **`embedding_dim`**.
2. Une couche récurrente (**`nn.RNN`** ou **`nn.LSTM`** avec **`batch_first=True`**) de dimension cachée **`hidden_size`**.
3. Une couche linéaire de sortie (**`nn.Linear`**) projetant chaque état caché h_t vers les scores (*logits*) des **`vocab_size`** caractères possibles.

Question 2.b. Complétez l'implémentation de la classe **`MusicModel`** :

```
class MusicModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim=256, hidden_size=512, rnn_type="lstm"):
        super().__init__()
        self.rnn_type = rnn_type.lower()
        self.embedding = nn.Embedding(__________, _____________)

        if self.rnn_type == "lstm":
            self.rnn = nn.LSTM(_____________, ___________, batch_first=True)
        elif self.rnn_type == "rnn":
            self.rnn = nn.RNN(_____________, ___________, batch_first=True)
        else:
            raise ValueError("rnn_type doit être 'rnn' ou 'lstm'")

        self.fc = nn.Linear(___________, __________)
```

```
    def forward(self, x, hidden_state=None):
        # x : (batch_size, seq_len)
        embeds = self.embedding(x)               # -> (batch_size, seq_len, embedding_dim)
        out, hidden_state = self.rnn(embeds, hidden_state) # -> out : (batch_size, seq_len,
hidden_size)
        logits = self.fc(out)                    # -> (batch_size, seq_len, vocab_size)
        return logits, hidden_state

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

✍️ À mettre dans votre rapport :

1. Calcul théorique des paramètres : Pour **`vocab_size = 95`**, **`embedding_dim = 256`** et **`hidden_size = 512`**, calculez à la main le nombre exact de paramètres de chacune des trois couches (**`embedding`**, **`rnn`**, **`fc`**) lorsque **`rnn_type="rnn"`**, puis lorsque **`rnn_type="lstm"`** (attention : dans PyTorch, les couches récurrentes possèdent deux vecteurs de biais b_{ih} et b_{hh} par porte !). Vérifiez votre calcul avec **`count_parameters`**. Pourquoi le bloc LSTM a-t-il exactement 4 fois plus de paramètres que le bloc RNN ?
2. Pourquoi est-il bien plus pertinent d'utiliser une couche **`nn.Embedding`** en entrée plutôt que de passer directement l'indice scalaire i dans {0, ..., V-1\} dans une couche linéaire 1 -> embedding_dim} ?
3. Quelle est la différence de structure entre la variable **`hidden_state`** retournée par **`self.rnn`** lorsque l'on utilise **`nn.RNN`** par rapport à **`nn.LSTM`** ? Expliquez le rôle respectif des deux états retournés par le LSTM.

Exercice 3 : Exercice 3 : Entraînement, Dynamique des Gradients et Comparaison RNN vs LSTM (~45mn)

Lors de la rétropropagation à travers le temps (BPTT) sur une séquence de 500 caractères, le gradient traverse 500 fois la même matrice de transition récurrente. Comme vu en cours, cela expose particulièrement le modèle aux problèmes de vanishing gradient et d'exploding gradient. Pour observer et contrôler ce phénomène, nous allons :

- Calculer et enregistrer la norme globale L2 des gradients dans TensorBoard à l'aide de **`torch.nn.utils.clip_grad_norm_`**.
- Implémenter l'Early Stopping pour sauvegarder automatiquement le meilleur modèle sur l'ensemble de validation.

Question 3.a. Complétez les fonctions d'évaluation et d'entraînement ci-dessous. Portez une attention particulière au redimensionnement (**`reshape`** / **`view`**) nécessaire avant le calcul de la **`CrossEntropyLoss`**, ainsi qu'à l'emplacement du *Gradient Clipping* :

```
@torch.no_grad()
def evaluate_model(model, val_loader, loss_fn, vocab_size):
    model.eval()
    total_loss, total_correct, total_tokens = 0.0, 0, 0

    for x, y in val_loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        logits, _ = model(x)

        # Aplatissement des dimensions (batch_size * seq_len, vocab_size) pour CrossEntropyLoss
        loss = loss_fn(logits.reshape(-1, __________), y.reshape(-1))

        total_loss += loss.item() * y.numel()
```

```
        preds = logits.argmax(dim=-1)
        total_correct += (preds == y).sum().item()
        total_tokens  += y.numel()

    return total_loss / total_tokens, total_correct / total_tokens


def train_model(
    rnn_type="lstm",
    subset_size=10000,
    num_epochs=15,
    batch_size=128,
    lr=5e-3,
    embedding_dim=256,
    hidden_size=512,
    max_grad_norm=None,
    patience=5,
):
    train_dataset = MusicDataset(train_data, subset_size=subset_size)
    val_dataset   = MusicDataset(val_data)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,  num_workers=2,
pin_memory=True)
    val_loader   = DataLoader(val_dataset,   batch_size=batch_size, shuffle=False, num_workers=2,
pin_memory=True)

    model = MusicModel(vocab_size, embedding_dim, hidden_size, rnn_type=rnn_type).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss()

    clip_tag = f"clip{max_grad_norm}" if max_grad_norm is not None else "noclip"
    run_name = f"{rnn_type}_sub{subset_size}_lr{lr}_{clip_tag}_{time.strftime('%H%M%S')}"
    writer = SummaryWriter(log_dir=f"runs_rnn/{run_name}")
    save_path = f"best_{rnn_type}.pth"

    best_val_loss = float("inf")
    patience_counter = 0
    global_step = 0

    for epoch in range(1, num_epochs + 1):
        model.train()
        total_train_loss, total_train_tokens = 0.0, 0
        epoch_grad_norms = []

        for x, y in tqdm(train_loader, desc=f"[{run_name}] Epoch {epoch}/{num_epochs}", leave=False):
            x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
            optimizer.zero_grad(set_to_none=True)
```

```
            logits, _ = model(x)
            loss = loss_fn(logits.reshape(-1, __________), y.reshape(-1))
            loss.backward()

            # Calcul de la norme des gradients (et clipping éventuel AVANT optimizer.step())
            if max_grad_norm is not None:
                grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=_____________)
            else:
                # Calcul de la norme sans modifier les gradients (max_norm=inf)
                grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=float("inf"))

            epoch_grad_norms.append(grad_norm.item())
            if global_step % 20 == 0:
                writer.add_scalar("GradNorm/step", grad_norm.item(), global_step)

            optimizer._____()

            total_train_loss += loss.item() * y.numel()
            total_train_tokens += y.numel()
            global_step += 1

        avg_train_loss = total_train_loss / total_train_tokens
        avg_grad_norm  = sum(epoch_grad_norms) / len(epoch_grad_norms)
        val_loss, val_acc = evaluate_model(model, val_loader, loss_fn, vocab_size)

        writer.add_scalar("Loss/Train", avg_train_loss, epoch)
        writer.add_scalar("Loss/Validation", val_loss, epoch)
        writer.add_scalar("Accuracy/Validation", val_acc, epoch)
        writer.add_scalar("GradNorm/Epoch_Avg", avg_grad_norm, epoch)
        writer.flush()

        print(f"Epoch {epoch:02d} | Train Loss: {avg_train_loss:.4f} | Val Loss: {val_loss:.4f} | "
              f"Val Acc: {val_acc:.4f} | Avg GradNorm: {avg_grad_norm:.3f}")

        # Early Stopping
        if val_loss = patience:
                print(f"Early stopping déclenché à l'époque {epoch}.")
                break

    writer.close()
    return model
```

✍️ **Expériences à réaliser en séance (15 époques, `subset_size=10000`) :** Lancez les 3 entraînements rapides suivants (~2 à 3 minutes chacun sur GPU) :

1. **Run 1 :** `rnn_type="rnn"`, `max_grad_norm=None` (RNN classique sans clipping)
2. **Run 2 :** `rnn_type="rnn"`, `max_grad_norm=1.0` (RNN classique avec clipping)
3. **Run 3 :** `rnn_type="lstm"`, `max_grad_norm=1.0` (LSTM avec clipping)

✍️ À mettre dans votre rapport :

1. **Manipulation des tenseurs :** Pourquoi doit-on écrire `logits.reshape(-1, vocab_size)` et `y.reshape(-1)` avant d'appeler `loss_fn` ? Que représente la valeur numérique de la dimension obtenue à la place du `-1` ?
2. **Ordre des opérations :** Pourquoi la fonction `clip_grad_norm_` doit-elle impérativement être appelée **après** `loss.backward()` mais **avant** `optimizer.step()` ?
3. **Analyse expérimentale (TensorBoard) :** Insérez dans votre rapport les courbes comparatives TensorBoard (`Loss/Validation`, `Accuracy/Validation` et `GradNorm/step`) des 3 runs :
    - Comparez l'évolution de la norme des gradients et la stabilité de la perte entre le **Run 1** et le **Run 2**. Quel problème théorique du RNN simple le *gradient clipping* corrige-t-il ici ?
    - Comparez les performances (vitesse de convergence, Loss et Accuracy de validation finales) entre le **Run 2 (RNN)** et le **Run 3 (LSTM)**. Expliquez mathématiquement (à l'aide des portes et de l'équation de mise à jour du *cell state* c_t vue en cours) pourquoi le LSTM parvient à capturer des dépendances plus longues dans la partition musicale.
4. **Réflexion sur la métrique :** Sachant qu'une partition de 250 caractères a été paddée jusqu'à 500 caractères avec des espaces `' '`, pourquoi l'`Accuracy/Validation` affichée est-elle artificiellement optimiste quant à la capacité réelle du modèle à prédire des notes de musique ? Quelle modification simple de `nn.CrossEntropyLoss` permettrait de ne pas compter le padding dans la perte ?

Exercice 4 : Exercice 4 : Génération musicale, Température et Job Slurm (~35mn)

Une fois le modèle LSTM entraîné, nous pouvons l'utiliser en mode **auto-régressif** pour composer de nouvelles partitions à partir d'une amorce (*prompt*, par exemple l'en-tête `"X:1\nT:Telecom Jig\nM:6/8\nL:1/8\nK:G\n"`).

Plutôt que de choisir systématiquement le caractère le plus probable (*greedy decoding* : argmax), qui enferme souvent le réseau dans des boucles répétitives, on échantillonne selon la distribution de probabilité modulée par une **température** T > 0 : P(y_t = c) = \frac{\exp(z_{t,c} / T)}{\sum_{c'} \exp(z_{t,c'} / T)} où z_t est le vecteur de logits produit par la couche linéaire au pas de temps t.

Étape 1 : Implémentation de la génération avec propagation de l'état caché

Question 4.a. Complétez la fonction `generate_music` ci-dessous. Observez comment on traite d'abord l'amorce pour initialiser l'état caché `hidden_state`, puis comment on ne passe plus qu'**un seul caractère à la fois** à chaque itération de la boucle :

```
@torch.no_grad()
def generate_music(model, start_sequence, output_length=300, temperature=1.0):
    model.eval()
    generated_seq = start_sequence

    # 1. Phase de pré-remplissage (lecture de l'amorce)
    input_indices = [char_to_index[c] for c in start_sequence]
    input_tensor = torch.tensor(input_indices, dtype=torch.long, device=device).unsqueeze(0) # (1,
len_prompt)

    # On récupère les logits et l'état caché (h, c) à la fin de l'amorce
    logits, hidden_state = model(input_tensor, hidden_state=None)

    # 2. Boucle de génération auto-régressive caractère par caractère
    for _ in range(output_length):
        # Logits du dernier pas de temps divisés par la température
```

```
        last_logits = logits[:, -1, :] / _____________
        probs = F._________(last_logits, dim=-1).squeeze(0)

        # Tirage aléatoire selon la distribution de probabilité
        next_index = torch.multinomial(probs, num_samples=1).item()
        next_char = index_to_char[next_index]
        generated_seq += next_char

        # Le caractère généré devient la seule entrée (1, 1) du pas de temps suivant !
        input_tensor = torch.tensor([[next_index]], dtype=torch.long, device=device)
        logits, hidden_state = model(input_tensor, hidden_state=_____________)

    return generated_seq
```

✍️ À mettre dans votre rapport :

1. **Efficacité algorithmique :** Dans la boucle **`for`** ci-dessus, nous passons uniquement le dernier caractère généré **`[[next_index]]`** de taille **`(1, 1)`** tout en fournissant **`hidden_state`** au modèle. Si, au lieu de conserver **`hidden_state`**, nous avions concaténé toute la séquence générée depuis le début et repassé la séquence entière à **`model(full_seq, None)`** à chaque itération, quelle aurait été la complexité temporelle totale pour générer N caractères ? Quelle est la complexité avec notre méthode ?
2. **Rôle mathématique de la température T :**
   - Que devient la distribution de probabilité softmax(z_t / T) lorsque T \to 0^+ ? À quelle stratégie de décodage cela correspond-il ?
   - Que devient cette distribution lorsque T \to +\infty ?
3. **Écoute et analyse qualitative :** À l'aide de votre meilleur modèle LSTM sauvegardé (ou après avoir lancé un entraînement plus complet à l'étape 2 ci-dessous), générez 3 partitions de 300 caractères à partir de l'amorce **`"X:1\nT:Irish Reel\nM:4/4\nL:1/8\nK:D\n"`** avec trois températures :
   - T = 0.2 (basse température)
   - T = 0.8 (température modérée)
   - T = 2.0 (haute température)

   Copiez les 3 partitions générées dans votre rapport et testez-les sur ABC Player. Commentez la validité syntaxique (respect des barres de mesure, des notes ABC) et la richesse musicale (répétitions vs chaos) de chacune des trois générations.

Étape 2 : Entraînement à plus grande échelle via `sbatch`

Question 4.b. Pour obtenir un générateur musical de meilleure qualité, lancez un job Slurm non interactif entraînant votre LSTM (avec **`hidden_size=1024`**, **`subset_size=50000`** ou **`None`**, **`max_grad_norm=1.0`**, **`num_epochs=30`**) et générant automatiquement les trois exemples musicaux (T dans {0.2, 0.8, 2.0}) à la fin de l'exécution dans le fichier de log.

```
#!/bin/bash
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH -J music-lstm
#SBATCH -o logs/%x-%j.out
```

```
#SBATCH -e logs/%x-%j.err

set -euo pipefail
mkdir -p logs

source ~/miniforge3/etc/profile.d/conda.sh
conda activate deeplearning

echo "Job $SLURM_JOB_ID sur $SLURM_NODELIST"
nvidia-smi

python -u train_music.py
```

✍️ **Action finale :** Soumettez ce script avec **`sbatch run_music.sh`**. **N'oubliez pas d'ajouter dans le dossier `TP4` de votre dépôt Git : votre script `train_music.py`, votre script `run_music.sh`, vos captures TensorBoard et votre `rapport.md` complet !**

Question 4.c. **(Bonus facultatif) Augmentation de données musicales :** En musique, transposer une mélodie d'un ou plusieurs demi-tons (ou d'une tonalité à une autre) conserve sa structure musicale tout en modifiant les caractères de notes et l'en-tête **`K:`**. Expliquez dans votre rapport comment vous pourriez implémenter une telle augmentation de données sur le format ABC et pourquoi cela réduirait le sur-apprentissage.

Remarque : N'oubliez pas de commiter vos dernières modifications (`rapport.md`, scripts Python, script `.sh`, extraits ABC générés et captures TensorBoard) dans votre dossier `TP4`, et de faire un push sur votre dépôt Git.